IE 206: Introduction to AI & ML

Perceptron and Feature Engineering

Manjesh K. Hanawal

Half Space

Definition of Halfspcae

- $h_{w,b}(x)$ represents an affine function, which is a weighted sum of the input vector $x \in \mathcal{X} = \mathcal{R}^d$.
- The class of affine functions in $\mathcal{R}^d$ as $L_d = \{h_{w.w_0} : w \in \mathcal{R}^d, w_0 \in \mathcal{R}\}$

$$h_{w,w_0}(x) = \langle w, x \rangle + w_0 = \sum_{i=1}^{d} w_i x_i + w_0$$

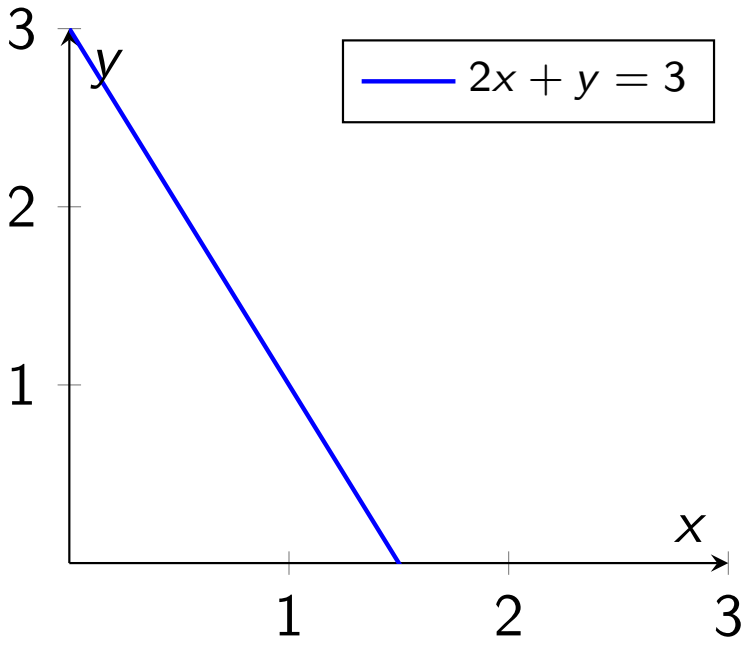
- The class of halfspaces (HS_d) is designed for binary classification problems,i.e., $\mathcal{Y} = \{-1, +1\}$. It is defined as:

$$HS_d = \text{sign} \circ L_d = \{x \mapsto \text{sign}(h_{w,w_0}(x)) : h_{w,w_0} \in L_d\}$$

Examples

$$\text{sign}(h_{w,w_0}(x)) = \begin{cases} +1 & \textit{if} \quad h_{w,w_0}(x) > 0 \\ -1 & \textit{otherwise} \end{cases} \tag{1}$$

This divides the space into two regions separated by the hyperplane defined by $\langle w, x \rangle + w_0 = 0$.



How to find the best hyperplane?

Perceptron Algorithm

- Inspired by biological neurons, a perceptron is the building block of neural networks.
- The perceptron takes a set of inputs, calculates a weighted sum, adds a bias term, and applies an activation function to produce an output.
- This mimics the behavior of a neuron in the brain, where the activation function decides whether the neuron "fires" or not.
- The idea of perception was developed to model decision-making processes similar to those in biological systems.

The Architecture

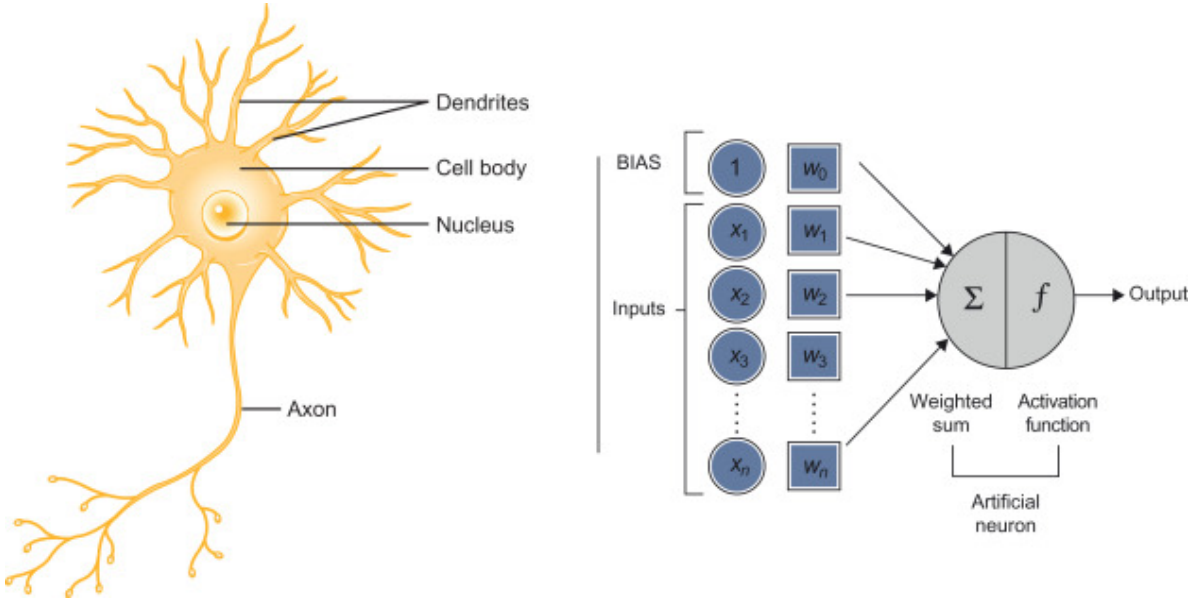


Figure: Perceptron : a node in neural network

1. A vector of inputs $x_1, x_2, \ldots, x_m$ is passed to the algorithm.
2. Weightings $w_1, w_2, \ldots, w_m$ are applied to each input, with a bias w_0 added.
3. A summation is performed: $w_0 + \sum_{i=1}^{m} x_i w_i$.
4. The sum is passed to an activation function g.
5. The activation function returns an output $\hat{y}$, determining the classification decision.

Perceptron Training Algorithm

Input: A training set $(x_1, y_1), \ldots, (x_m, y_m)$

Initialize: $w^{(1)} = (0, \ldots, 0)$

For $t = 1, 2, \ldots$ **do**

If $\left(\exists i \text{ s.t. } y_i \langle w^{(t)}, x_i \rangle \leq 0\right)$ **then**

$w^{(t+1)} = w^{(t)} + y_i x_i$

Else Output $w^{(t)}$

- The algorithm iteratively reduces the number of misclassifions.
- For separable data, it convergence to a separating hyperplane.
- update of the Perceptron guides the solution to be "more correct" on the i'th example.

Guarantee of Convergence

Theorem: Assume that the dataset $S : \{(x_1, y_1), \ldots, (x_m, y_m)\}$ is separable. Define:

$$B = \min\{\|w\| : \forall i \in [m], y_i \langle w, x_i \rangle \geq 1\} \quad \textit{and} \quad R = \max_i \|x_i\|$$

The Perceptron algorithm stops after at most T iterations:

$$T \leq (RB)^2.$$

.

- When the algorithm stops, the resulting weight vector $w^{(t)}$ satisfies:
$$\forall i \in [m], \; y_i \langle w, x_i \rangle > 0.$$
- The convergence rate depends on B (margin of separation) and R (norm of samples)
- Highlights the importance of feature scaling in practical applications.

Feature engineering: normalizing and centralizing data

Normalization: This process scales the feature values to a range, typically between 0 and 1 or -1 and 1. This is particularly useful when features have different units or scales.
Centralization: This involves subtracting the mean of the feature values from each value, effectively centering the data around zero. The transformation is given by:

$$\phi(x) = \frac{x - \bar{x}}{\sigma}$$

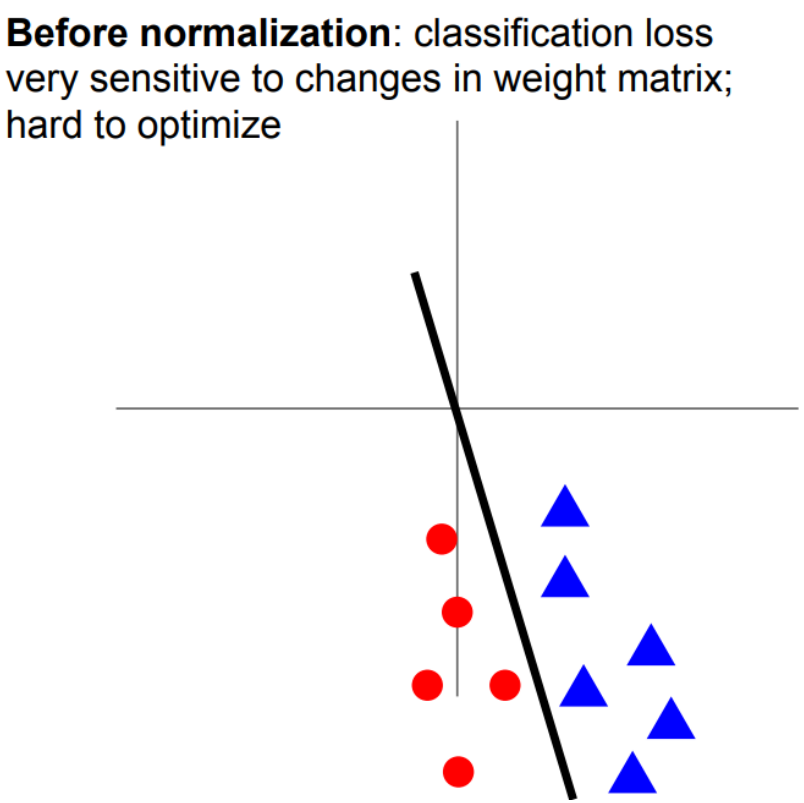
where:

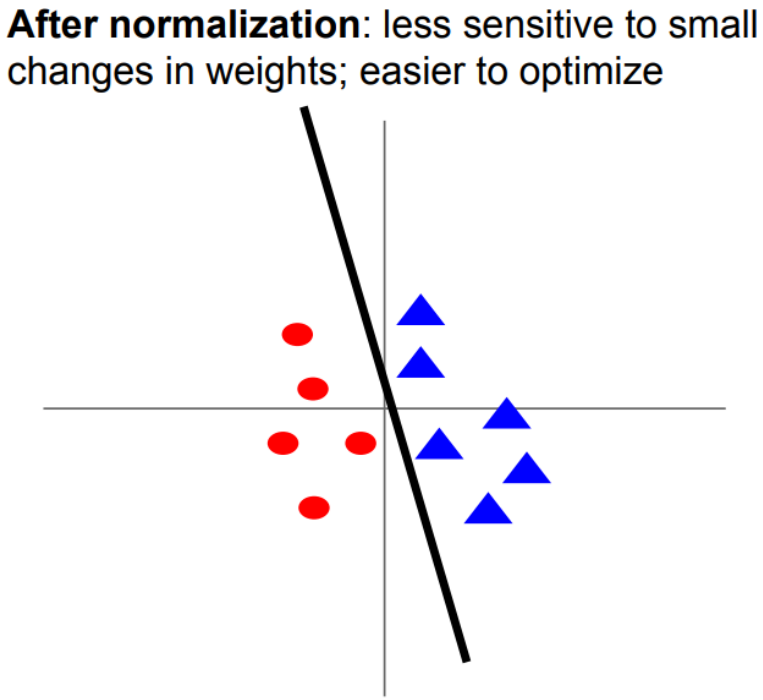
- $\bar{x}$: Mean of the feature values x_i,
- σ: Standard deviation of x_i.

This results in feature values with a mean of 0 and a standard deviation of 1, often referred to as "z-scores."

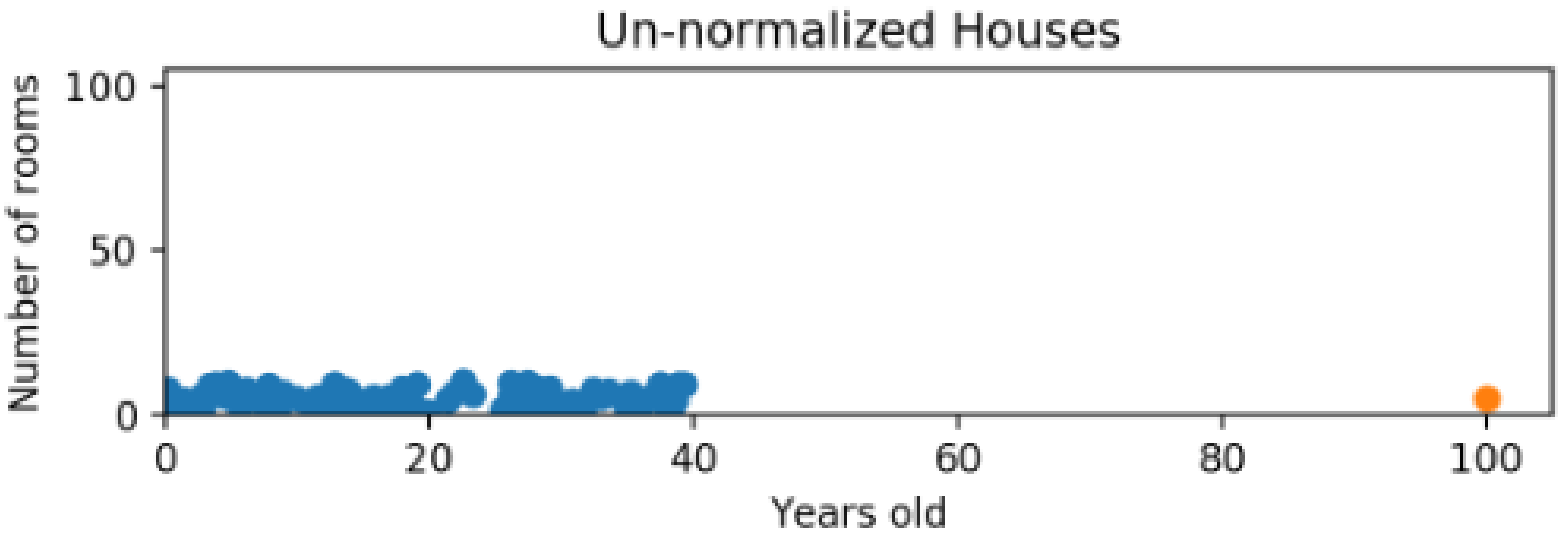
Normalized vs Un-normalized

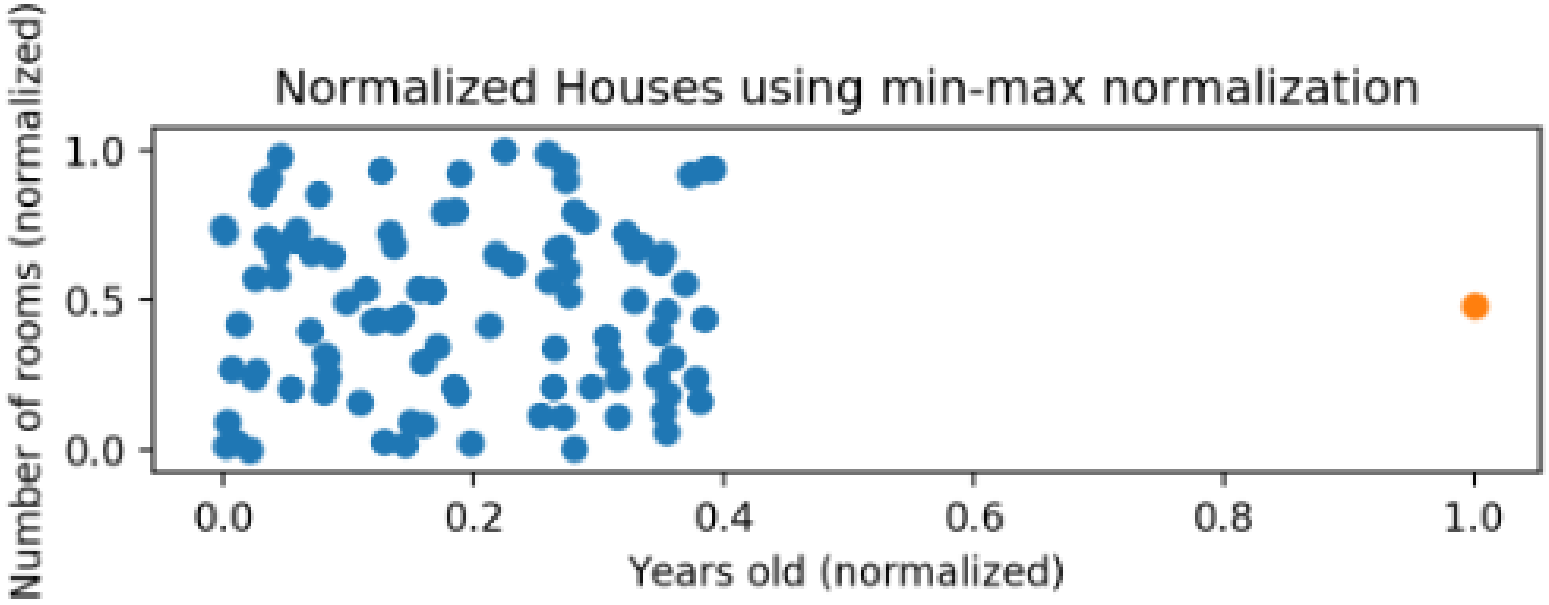
Normalization ensures that all features contribute equally, improving convergence rates for optimization algorithms. Without normalization, large-value features may dominate smaller-value features, skewing results.





Example

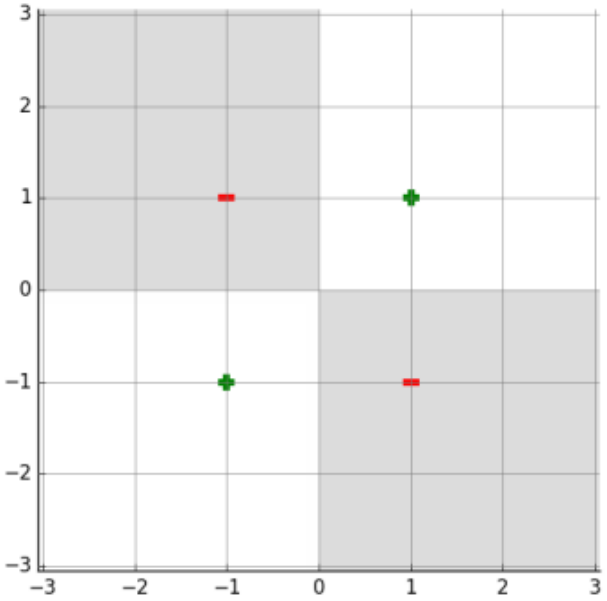

Un-normalized Houses
Number of rooms
100
50
0
0
20
40
60
80
100
Years old

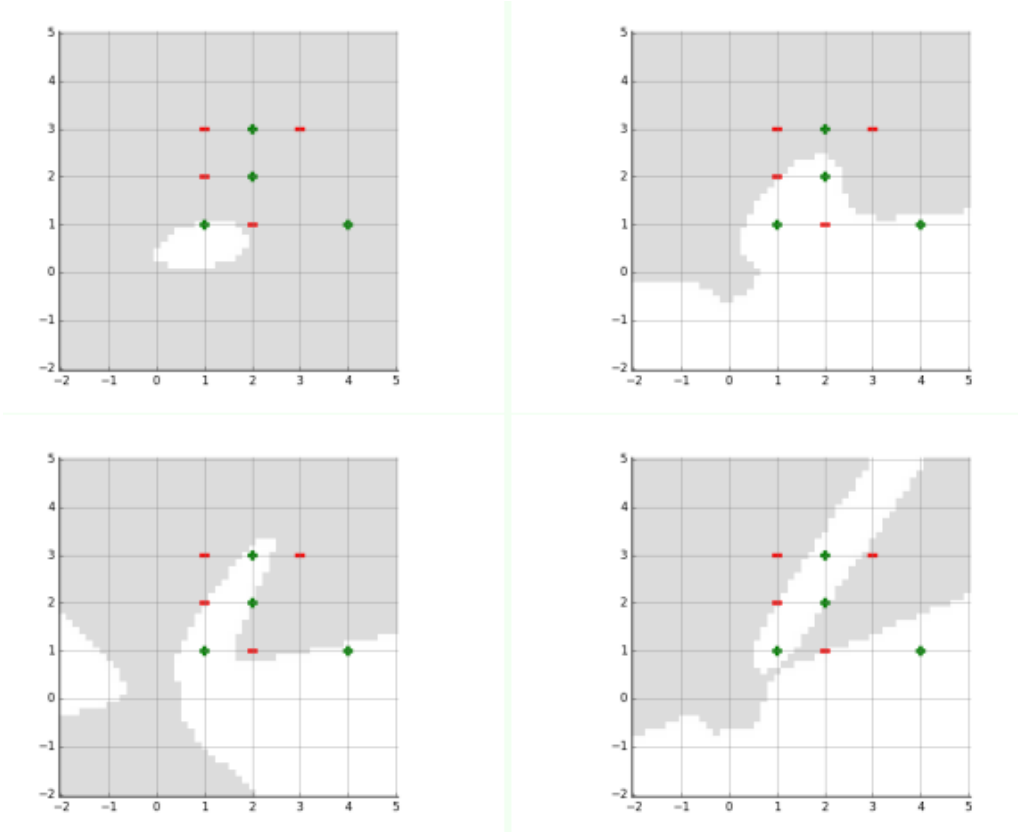

Normalized Houses using min-max normalization
Number of rooms (normalized)
1.0
0.5
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Years old (normalized)

Feature Representation: Generating More Features

The XOR problem illustrates how higher-dimensional transformations make data linearly separable:

- Input: Non-linearly separable 2D data.
- Transformation: $\phi((x_1, x_2)) = (1, x_1, x_2, x_1^2, x_1x_2, x_2^2)$.
- Result: The higher-dimensional space allows the data to be linearly separable.



Polynomial Basis

- **Polynomial Basis**: If the original features are numerical, one systematic way to create a new feature space is to use a polynomial basis. For example, if you have a 1D feature x, you can create new features like x^2, x^3, etc. This increases the dimensionality of the data, making it easier to find a linear separator in the higher-dimensional space.